PeerForge — User Manual

How to use the product, and how to test that it works. Verified against a running instance, 2026-08-05.

Part 1 · Using PeerForge

1.1 What it does for you

You have a paper, a thesis chapter or a grant application. You want to know what a review panel will say about it **before** they say it. PeerForge runs that panel: several reviewers with different remits read your work, prepare privately, then argue about it in front of you. You leave with a transcript, a list of the hardest questions you will be asked, a readiness score, and a certificate you can share.

You do not need an API key. The server holds one.

1.2 The five-minute path

1. **Open the app.** No login — you land in a workspace.
2. **New session.** Give it a title and paste your abstract or problem

statement into the problem-statement field.

1. **Upload your paper.** Wait for processing to reach `complete` — the page

shows this. Text extraction, chunking and embedding all happen here.

1. **Build the panel.** Two ways:

- **Conversational** (`/setup/chat`) — describe what you want in prose:

"a statistician who won't let a weak design pass, a clinical psychology expert, and an examiner who decides publishability, two rounds". Review the proposal and apply it.

- **Manual** (`/setup`) — pick reviewers from templates and set the rounds.

1. **Run preflight.** Each reviewer reads your work, searches the literature,

and writes a private memo. Click **View Prep Pack** on any reviewer to read what they prepared and which sources they consulted.

1. **Enter the room.** Press **Start**, then **Next Turn** for each

contribution — or switch on autonomous mode and watch.

1. **When it ends** — generate the summary, the defence questions and the

assessment, then issue a certificate.

1.3 Writing a good problem statement

This is the single highest-leverage thing you control. It drives the web search, the retrieval query and every reviewer's framing.

Weak: "My paper about mindfulness."

Strong: "A randomised trial of a six-week smartphone mindfulness app on undergraduate exam anxiety (n=40) reports a large effect, with allocation performed by the first author who also delivered the intervention, no active control and no preregistration."

The strong version names the design, the sample, the claim **and** the things you already suspect are weak. Reviewers will find more when you are honest about what is fragile.

1.4 Choosing a panel

Six reviewer lanes exist. Mix them — a panel of three methodologists finds one kind of problem three times.

Lane	Presses on
Methodology professor	Design, sampling, controls, validity threats, statistics
Domain expert	Domain correctness, novelty, related work, construct validity
Skeptical reviewer	Whether claims exceed evidence; generalisation limits
External examiner	Whether it is defensible for publication, and what must change
Advisor	Whether the work delivers on its own stated goals
Friendly professor	Clarity, structure, terminology, communication

Three reviewers over two rounds is a good default: six turns, roughly five minutes, enough for each lane to open and then respond.

1.5 The room

Control	What it does
Start	Opens the session
Next Turn	The next reviewer speaks (or Ctrl/Cmd+Enter)
Pause / Resume	Halts and continues
⚡ **Resume Auto**	The panel advances itself on a timer
+2 Rounds	Extends the session
End Meeting	Closes it, so the summary can be produced

Intervene lets you interject as the researcher — clarify a method, push back on a misreading, redirect the panel. Reviewers will pick it up on their next turn.

1.6 Reading the output

- **Transcript** — the argument as it happened.
- **Summary** — grounded in the real transcript. If the transcript is empty

it refuses to invent one.

- **Defence questions** — 15 viva-style questions, each with the excerpt from

your document that prompted it. Answer them in the app; the assessment updates.

- **Assessment** — 10 scored dimensions, with a stated basis showing what it

had to work with (`has_profile`, `answer_count`, `message_count`). Its honesty cuts both ways: answering candidly about a real flaw can **lower** your score.

- **Certificate** — sha256 over the scores and the ordered evidence, signed.

Share the `/verify/{id}` link.

1.7 Understanding certificate verdicts

Verdict	Meaning
VALID	Signature and hash check out; the evidence is unchanged
SUPERSEDED	Authentic, but the session has moved on since it was issued. Re-issue to certify the current state
INVALID	Signature or hash fails — the certificate has been altered

`SUPERSEDED` is not an accusation. It means the work progressed.

1.8 What the system will not do

- It will not cite a page or section of a document you did not upload. If it

tries, the text is replaced with `[source not provided]`.

- It will not cite a section your document does not contain.
- It will not summarise a session that never happened.
- It will not ask you for an API key.

Part 2 · Testing PeerForge

For anyone verifying the system — QA, a technical evaluator, or a developer after a change.

2.1 Automated tests

```
cd apps/api && source venv/bin/activate
python -m pytest tests/ -q
```

Expect **440 passed, 4 skipped**.

```
cd apps/web
npx tsc --noEmit            # typecheck
npm run lint                # includes the key-gate and accessibility checks
node scripts/check-key-gates.js
```

Expect `OK: no generation gate depends on a browser-held key`.

> ⚠ **The backend suite writes to the same database as the app** and leaves its fixtures behind — running it after seeding demo data adds hundreds of junk sessions. Seed **after** testing, or point the tests at a separate database.

2.2 Manual end-to-end script

Set once:

```
API=http://localhost:8000
H='Authorization: Bearer anonymous'
WS=$(curl -s $API/me/workspaces -H "$H" | python3 -c 'import sys,json;print(json.load(sys.stdin)["active_workspace_id"])')
```

Create a session, with a problem statement:

```
D=$(curl -s -X POST $API/debates -H 'Content-Type: application/json' -H "$H" \
  -d '{"workspace_id":"'$WS'","title":"E2E","problem_statement":"A randomised trial of a six-week mindfulness app on exam anxiety (n=40) reports a large effect with no active control and no pr
  | python3 -c 'import sys,json;print(json.load(sys.stdin)["debate_id"])')
```

 Check: `GET /debates/$D` returns `policy_config.problem_statement` populated. If it is absent, the field is being dropped.

Upload and embed:

```
curl -s -X POST $API/debates/$D/materials/upload -H "$H" \
  -F "files=@paper.txt" -F "is_primary=true"
curl -s "$API/debates/$D/materials/status" -H "$H"      # wait for {"complete": 1}
curl -s -X POST $API/debates/$D/materials/embed -H "$H" \
  -H 'Content-Type: application/json' -d '{}'
```

 Check: `memory_chunks` has rows for this debate with non-empty `chunk_text`.

Preflight, and confirm web research and citation:

```
curl -s -X POST $API/debates/$D/preflight/start -H "$H" \
  -H 'Content-Type: application/json' -d '{}'
```

 Check the prep pack metadata:

Field	Expected
`web_research_status`	`ok` (or a named reason — never a bare false)
`web_search_urls`	5 entries
`web_sources_cited`	≥ 2

If the status is `not_configured`, `TAVILY_API_KEY` is unset — that is configuration, not a fault, and the UI will say so.

Run the review:

```
curl -s -X POST $API/debates/$D/start -H "$H" -H 'Content-Type: application/json' -d '{}'
for i in 1 2 3 4 5 6; do
  curl -s -X POST $API/debates/$D/turn/next -H "$H" -H 'Content-Type: application/json' -d '{}'
done
```

Finish the chain:

```
curl -s -X POST $API/debates/$D/end          -H "$H" -H 'Content-Type: application/json' -d '{}'
curl -s -X POST $API/debates/$D/summarize    -H "$H" -H 'Content-Type: application/json' -d '{}'
curl -s -X POST $API/debates/$D/analyze-research -H "$H" -H 'Content-Type: application/json' -d '{}'
curl -s -X POST $API/debates/$D/defense-questions/generate -H "$H" -H 'Content-Type: application/json' -d '{}'
curl -s -X POST $API/debates/$D/assessment/generate -H "$H" -H 'Content-Type: application/json' -d '{}'
curl -s -X POST $API/debates/$D/certificate/issue -H "$H" -H 'Content-Type: application/json' -d '{}'
```

2.3 Measuring transcript quality

This is the check that matters most, and the one a casual pass will miss.

```
import json, sys; sys.path.insert(0, '.')
from src.services.transcript_quality import analyse, Turn, Thresholds

events = json.load(open('events.json'))
turns = [Turn(speaker=e['payload'].get('agent_name'), text=e['payload'].get('text',''))
         for e in events if e.get('type') == 'agent_message']
r = analyse(turns)
print(r.restatement_rate, r.opener_template_rate, r.failures(Thresholds()))
```

Metric	Pass	Means
`restatement_rate`	≤ 0.5	Turns restating a point already made
`opener_template_rate`	≤ 0.5	Turns opening by deferring instead of leading
`placeholder_rate`	≤ 0.2	`[source not provided]`, `@Name`

`self_similarity` and `role_differentiation` are reported but **do not gate** — measured across ten transcripts their good and bad ranges overlap completely.

2.4 Adversarial tests

These are the checks that find real regressions.

Fabricated citations. Create a session with **no** document, give three reviewers system prompts ordering them to cite pages and tables in every paragraph, run six turns.

 Pass: zero locators (`p. 12`, `Section 2.1`, `Table 3`) in the transcript. Survivors appear as `[source not provided]`.

Contradicted citations. Upload a paper with Sections 1–4 only, instruct reviewers to cite Sections 5–9 and Table 7, run six turns.

 Pass: zero of those reach the transcript; legitimate citations to Sections 1–4 survive untouched.

Key gates. Reintroduce a browser-key gate under any name:

```
const k = keyStore.getKey();
if (!k) { setError('Add your key'); return; }
```

 Pass: `node scripts/check-key-gates.js` exits non-zero and names it.

Room controls. On a running session, each of these returns 200 and moves the state: `start` → running, `pause` → paused, `resume` → running, `PATCH /extend` with `{"extend_rounds":2}`, `turn/next`, `end` → ended.

> Note: `/extend` expects `extend_rounds`, not `additional_rounds`. Sending the wrong field returns a 400 that looks like a bug and is not.

2.5 Surface check

```
for ep in "" /events /summary /assessment /certificate /provenance /quality \
          /action-items /research-profile /defense-questions /materials/status; do
  echo "$ep $(curl -s -o /dev/null -w '%{http_code}' $API/debates/$D$ep -H "$H")"
done
```

All 200 on a completed session. Three endpoints return 404/422 legitimately: `/document` when no document artefact exists, `/readiness-report` before you POST one, and `/memory/preview` without

its required query parameter.

2.6 Troubleshooting

Symptom	Cause	Fix
"Web research was not performed"	No `TAVILY_API_KEY`	Set it server-side; the panel names the reason
Prep pack fails to load	—	Fixed; if it recurs, check `GET /agent-knowledge/{id}` directly
Next Turn disabled	A browser-key gate	Run `check-key-gates.js`
`/extend` returns 400	Wrong field name	Use `extend_rounds`
Summary refuses	Debate not ended, or no transcript	End the session first
Certificate 404 on verify	Content hash changed; never issued	`POST /certificate/issue` first
Hundreds of unexpected sessions	The test suite ran against this DB	Clean, then seed after testing

2.7 Rebuilding the demo institution

```
cd apps/api && source venv/bin/activate
python scripts/seed_demo_org.py     # Northgate University, 4 courses, 8 members
```

Papers with deliberately planted flaws live in `scripts/demo_papers/` — unconcealed allocation, missing active control, thresholds tuned on the reported data, train/test leakage, untested parallel-trends assumptions. Good material for verifying reviewers find real problems.